

# OpenCart - Capstone Project

*QA automation capstone scope document using Playwright with JavaScript.*

| Field               | Details                                                                                                          |
|---------------------|------------------------------------------------------------------------------------------------------------------|
| Project type        | E-commerce website functional testing capstone                                                                   |
| Selected website    | OpenCart Demo Store                                                                                              |
| Website URL         | <a href="https://naveenautomationlabs.com/opencart/">https://naveenautomationlabs.com/opencart/</a>              |
| Automation stack    | Playwright with JavaScript                                                                                       |
| Minimum requirement | 8 services, 15 test cases per service, total 120 test cases                                                      |
| Testing focus       | Frontend user flows, form validations, cart and checkout behavior, account-related pages, and reporting evidence |

## Project Overview

The primary objective of this capstone is to design and execute a comprehensive QA automation suite for the OpenCart demo application. This project focuses strictly on quality assurance methodologies and test execution.

OpenCart serves as an ideal environment for this testing framework because it fully simulates a live, data-driven e-commerce platform. By validating essential consumer journeys, such as account registration, product searches, cart manipulation, and the end-to-end checkout pipeline, the project will effectively demonstrate real-world testing capabilities and ensure functional stability across the application, the site is suitable for a capstone project that requires at least 120 total test cases.

## Reason For Website Selection

- It is a public demo e-commerce application and can be accessed without building a website locally.
- The application's extensive feature set naturally breaks down into the 8 distinct, testable services required to fulfill the capstone criteria.
- It offers genuine e-commerce functionality, allowing the automation framework to simulate realistic consumer behaviors from initial product discovery through final order placement and account management.
- It can be tested using Playwright with JavaScript, matching the training technology stack.
- The site is reset regularly, so test data can be planned as disposable dummy data.

## Project Objectives

- Prepare a complete test plan for the selected website.
- Create a minimum of 120 test cases across 8 services.
- Cover positive, negative, validation, navigation, and regression scenarios.
- Automate the planned tests using Playwright with JavaScript.
- Generate execution evidence through test reports, screenshots, and traces wherever required.
- Present the project as a QA/testing capstone, not as a development project.

## Scope Of Testing

The scope is limited to the public frontend demo store. The project will focus on customer-facing functionality and avoid any activity that requires real payment, real personal information, or backend administration.

| In Scope                                                                             | Out Of Scope                                                                       |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Authentication:</b> Registration, login/logout, and account navigation.           | <b>Financials:</b> Real payment processing and actual order fulfillment.           |
| <b>Catalog:</b> Product searches, category browsing, sorting, and detail pages.      | <b>Backend:</b> Database validations and OpenCart admin panel modifications.       |
| <b>Transactions:</b> Cart manipulation, product comparisons, and checkout UI flows.  | <b>Non-Functional:</b> Performance, security, stress, or load testing.             |
| <b>Fulfillment:</b> Address book updates, shipping steps, and order history logs.    | <b>Real Data:</b> Testing with actual customer PII or real credit card details.    |
| <b>Support &amp; API:</b> Contact forms, footer links, and internal API validations. | <b>Environment:</b> Altering server configurations, product pricing, or core code. |

## Service-Wise Test Case Distribution

The capstone requirement will be met by preparing 15 test cases for each of the following 8 services. This gives a planned total of 120 test cases irrespective of browser selection.

| No. | Service / Module   | Testing coverage                                                                  | Count |
|-----|--------------------|-----------------------------------------------------------------------------------|-------|
| 1   | Authentication     | Register, login, invalid credentials, logout, and password recovery.              | 15    |
| 2   | Product Management | Search functionality, category navigation, sorting, product details, and reviews. | 15    |
| 3   | Shopping Cart      | Add to cart, update quantity, remove item, subtotal calculation, and mini-cart.   | 15    |

|   |                      |                                                                                      |    |
|---|----------------------|--------------------------------------------------------------------------------------|----|
| 4 | Checkout and Payment | Guest checkout, registered checkout, billing/shipping steps, and order confirmation. | 15 |
| 5 | User Profile         | Edit account info, change password, order history, and newsletter subscription.      | 15 |
| 6 | Address and Shipping | Add/edit/delete address, default address behavior, and address validation.           | 15 |
| 7 | Customer Support     | Contact Us form, product returns, site map, and footer link navigation.              | 15 |
| 8 | Internal API         | API status checks, login payloads, cart manipulation via API, and response times.    | 15 |

## Detailed Module Coverage

| Module               | Detailed planned coverage                                                                                                                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication       | Verify successful registration with dummy data, duplicate email handling, login success/failure scenarios, forgotten password retrieval, and secure logout behaviors.                                             |
| Product Management   | Verify keyword search, empty search handling, category navigation, list/grid view toggles, sorting by price/name, product detail verification (price, brand), and submitting product reviews.                     |
| Shopping Cart        | Verify adding items from various pages, updating item quantities, removing items via mini-cart and main cart, subtotal/tax calculations, and coupon code application.                                             |
| Checkout and Payment | Verify end-to-end guest and registered user checkout flows, mandatory billing/shipping fields, flat rate shipping, cash on delivery (COD) selection, Terms & Conditions validation, and final order confirmation. |
| User Profile         | Verify updating account information, changing passwords, viewing order history details, checking reward points, and managing newsletter subscriptions.                                                            |
| Address and Shipping | Verify adding new addresses, editing existing ones, deleting non-default addresses, state/country dropdown dependencies, and default address behavior during the checkout flow.                                   |
| Customer Support     | Verify Contact Us form submissions (valid/invalid data), product return requests, navigation to footer links (Privacy Policy, About Us), and Site Map accessibility.                                              |
| Internal API         | Verify HTTP status codes for core endpoints, validate login authentication payloads, test cart manipulation via API requests, and monitor API response times.                                                     |

## Types Of Testing Planned

| Testing type         | Purpose in capstone                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Functional testing   | Validate that each core OpenCart feature behaves according to the defined user journeys.                                          |
| Validation testing   | Ensure proper handling of mandatory checkout fields, blank form submissions, incorrect data formats, and UI warning messages.     |
| Negative testing     | Confirm the application safely restricts unauthorized access and gracefully handles invalid inputs without crashing.              |
| Navigation testing   | Verify the accuracy of all primary menu links, breadcrumb trails, footer URLs, and cross-page redirects.                          |
| Regression testing   | Continuously re-execute the automated Playwright suite to guarantee that test data changes do not break previously passing flows. |
| UI assertion testing | Assert the visibility and correctness of critical on-screen elements like product titles, cart subtotals, and order success text. |

## Automation Approach

The automation framework will be built using Playwright with JavaScript. The suite will be organized strictly by service to guarantee that each OpenCart module can be maintained and executed independently.

- **Core Runner:** Utilize Playwright Test as the primary execution engine for the suite.
- **Modular Specs:** Construct separate .spec.js files for each major OpenCart service, such as authentication, cart, checkout, and user profile.
- **Resilient Locators:** Prioritize stable, user-centric locators like roles, labels, text placeholders, and visible UI elements to minimize script flakiness.
- **Strict Assertions:** Implement hard assertions to constantly verify page URLs, success messages, product titles, cart totals, and final order confirmations.
- **Reusable Setup:** Leverage Playwright hooks and helper functions to handle repetitive prerequisites, like user logins or adding baseline products to the cart.
- **Data Independence:** Keep test data entirely separate from automation logic, allowing dummy names, emails, and addresses to be swapped out seamlessly.

## Planned Project Structure

```
Playwright-Capstone-Project/
|--- .gitignore
|--- package.json
|--- playwright.config.js
|
|--- tests/
|   |--- authentication.spec.js
|   |--- productManagement.spec.js
|   |--- shoppingCart.spec.js
|   |--- checkoutPayment.spec.js
|   |--- userProfile.spec.js
|   |--- addressShipping.spec.js
|   |--- customerSupport.spec.js
|   |--- internalApi.spec.js
|
|--- pages/
|   |--- BasePage.js
|   |--- LoginPage.js
|   |--- RegisterPage.js
|   |--- ProductPage.js
|   |--- CartPage.js
|   |--- CheckoutPage.js
|   |--- AccountPage.js
|
|--- test-data/
|   |--- users.json
|   |--- addresses.json
|
|--- utils/
|   |--- apiHelper.js
|   |--- randomDataGen.js
|
|---playwright-report/
```

## Test Data Strategy

The OpenCart platform is a public demo environment, all test execution will rely exclusively on dummy data to ensure safe, repeatable test runs.

- **Synthetic User Info:** Generate unique dummy emails, names, and phone numbers dynamically to prevent duplicate account registration errors during auth testing.
- **Zero Real PII:** Strictly avoid the use of actual customer data, personal information, or real credit card details during the checkout and payment flows.

- **Existing Catalog Utilization:** Leverage the native products already populated in the OpenCart demo rather than attempting to inject new items into the database.
- **Reusable Data Objects:** Centralize test data payloads—like standard billing addresses, shipping details, and contact form inputs within the test-data/ directory for easy maintenance.
- **Environment Reset Handling:** Design the automation scripts to be resilient against periodic database resets by creating fresh user accounts and carts at the start of required test runs.

## Entry And Exit Criteria

| Entry criteria                                                          | Exit criteria                                                                             |
|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| The OpenCart demo store is live and major pages are accessible.         | All 120 test cases are fully documented across the 8 chosen services.                     |
| The selected 8 testing modules and target scenarios are finalized.      | Playwright automation scripts are written and execute successfully for the planned scope. |
| The Playwright + JavaScript environment and POM structure are set up.   | The final Playwright HTML execution report is generated and reviewed.                     |
| Disposable dummy test data objects are prepared.                        | Any failing OpenCart flows are properly recorded with evidence.                           |
| The browser execution approach (e.g., Chromium, Firefox) is configured. | Final capstone files are organized for submission.                                        |

## High-Level Test Scenarios

| Service              | Example high-level scenarios                                                                                                                |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication       | Verify successful and invalid registration, successful and failed login attempts, secure logout, and account recovery access.               |
| Product Management   | Verify that customers can browse categories, accurately search for items, view precise product details, and use sorting filters.            |
| Shopping Cart        | Verify that items can be added or removed, quantities can be updated, and cart subtotals/taxes calculate correctly.                         |
| Checkout and Payment | Verify guest and registered user checkout, required billing/shipping steps, payment method selection, and order confirmation messages.      |
| User Profile         | Verify account detail updates, password change validations, order history visibility, and newsletter subscription toggles.                  |
| Address and Shipping | Verify address field validations, country/state dropdown behavior, saved address selection, and default address logic.                      |
| Customer Support     | Verify Contact Us form behavior, product return submissions, footer link navigation, and site map accessibility.                            |
| Internal API         | Verify HTTP status codes for core endpoints, validate authentication payloads, and ensure API response times meet performance expectations. |

## Defect Reporting Plan

If any test case fails during manual verification or automation execution, it will be recorded clearly so the issue can be understood without rerunning the full test immediately.

| Defect field        | Details to capture                                                          |
|---------------------|-----------------------------------------------------------------------------|
| Defect ID           | A unique tracking identifier (e.g., OC-BUG-001).                            |
| Module              | The specific service where the failure occurred (e.g., Checkout & Payment). |
| Summary             | A one-line description of the encountered problem.                          |
| Steps to reproduce  | The exact, numbered sequence of actions required to trigger the bug.        |
| Expected result     | What should have happened                                                   |
| Actual result       | What actually happened                                                      |
| Evidence            | Screenshot, trace, report link, or error message                            |
| Severity / priority | Impact and urgency of the issue                                             |
| Status              | Open, fixed, retest, or closed                                              |

## Reporting And Evidence

- Playwright HTML report will be used as the main execution report.
- Screenshots will be captured for important failed scenarios or final evidence if required.
- Trace files can be used to debug failed tests and show step-by-step execution.
- Final summary will include total tests planned, total tests automated, pass count, fail count, and pending items.

## Risks And Mitigation

| Risk                                                | Mitigation                                                                      |
|-----------------------------------------------------|---------------------------------------------------------------------------------|
| Demo site resets regularly                          | Use fresh dummy data and avoid depending on old registered accounts.            |
| Product names or prices may change                  | Use flexible assertions and update test data if visible website data changes.   |
| Public site may be slow or temporarily unavailable  | Use Playwright timeouts carefully and rerun after confirming site availability. |
| Some checkout paths may need specific product types | Select products that support predictable cart and checkout behavior.            |
| Automation locators may break if UI changes         | Prefer stable user-facing locators and keep selectors organized.                |

## Assumptions

- The capstone will run against the public frontend demo store at <https://naveenautomationlabs.com/opencart/>
- Only dummy test data will be used.
- The primary automation framework will be Playwright with JavaScript.
- The 120 test cases will be counted service-wise, not browser-wise.
- If the website resets or changes, affected test cases will be updated with trainer approval.

## Planned Capstone Flow

1. Finalize website and service-wise testing scope.
2. Prepare 120 test cases across 8 services.
3. Review and improve test cases based on trainer feedback.
4. Create Playwright with JavaScript project structure.
5. Automate tests service by service.
6. Execute tests, review failures, and fix automation issues.
7. Prepare final report, screenshots/traces, and submission folder.



## Final Deliverables

| Deliverable                   | Description                                                                                 |
|-------------------------------|---------------------------------------------------------------------------------------------|
| Capstone testing plan         | This document explaining website selection, scope, services, approach, and expected output. |
| Test case document            | Service-wise list of 120 test cases with steps, expected results, priority, and status.     |
| Playwright automation project | JavaScript test scripts organized by service/module.                                        |
| Test data files               | Dummy data required for registration, login, address, checkout, and forms.                  |
| Execution report              | Playwright HTML report showing test run results.                                            |
| Evidence folder               | Screenshots, traces, or notes for important passed/failed scenarios.                        |
| Final summary                 | Brief project summary with coverage, execution result, known issues, and conclusion.        |

## References

- OpenCart demo store frontend: <https://naveenautomationlabs.com/opencart/>
- Playwright documentation will be used later during automation setup and script execution.

## Conclusion

The selected website and planned scope are suitable for the capstone requirement because the application has enough independent services to prepare 120 test cases and enough realistic e-commerce behavior to demonstrate QA automation skills. The project will be completed using Playwright with JavaScript, matching the training stack and keeping the work focused on testing activities.